안녕하세요, 이세령입니다.

1999. 03. 09

srlimvp@gmail.com / 01038870560

https://github.com/Hediar

https://velog.io/@hediar

자동화와 구조 개선으로 운영 효율과 안정성을 높인 서비스 전주기 경험

플랫폼 개발팀(60명 중 20명)에서 IoT 수자원/시설물 서비스를 프론트, 백엔드 개발부터 데이터 관리 및쿼리튜닝, 운영업무 자동화 등 개발 전주기를 경험했습니다.

Cron+Slack, ExcelJS, Playwright 기반 **자동화로 운영 업무 시간을 50% 이상 절감**했고, **서비스 리뉴얼과 구조 개선**으로 유지보수 대응 시간을 20% 단축했습니다. 또한 **UI/UX 개선**을 통해 문의 및 오류를 약 80% 줄였으며, 검색 성능을 14초에서 5초로 개선하고 빌드 속도를 약 10% 향상시키는 등 운영 체감 지표를 지속적으로 개선했습니다.

모니터링 시스템을 구축해 수동 터널링 없이 오류 원인을 확인할 수 있는 환경을 마련했고, **데이터 수집 → 가공 → 저장** 전 과정을 관리하며 안정적인 **데이터 플로우 운영**에 기여했습니다.

경력

그렉터

풀스택 엔지니어
2023. 11. 03 - (재직 중)

IoT 플랫폼
개발자 약 20명
OS: Linux Debian

외부활동 및 학력

토스 Learner's High 2기

2025.12 - 2026.01

내일배움캠프 6기 React 트랙

2023.05 - 2023.09

한경대학교

2021 - 2023 (컴퓨터공학과)

원광대학교

2019 - 2021 (컴퓨터소프트웨어공학과)

주요 기술

NextJS
Vue
Spring boot
Node.js
JavaScript
TypeScript
Linux
Opensearch/Elasticsearch
React
Postgresql

자격증

정보처리기사

2025.09

SQLD

2026.03

POPO(AI 기반 포트폴리오 챗봇) 2026. 02 ~ 03

Project

개요

이력서를 기반으로 사용자의 질문에 답변하는 RAG 기반 AI 챗봇

목표

- **RAG 패턴** 구현 : PostgreSQL + pgvector로 벡터 데이터베이스 구축
- AI 대화 시스템 구축 : OpenAI Chat API를 통합하여 1인칭 시점의 자연스러운 응답 생성
- Spring Boot 기반 RESTful API 개발
- **새로운 기술 스택 학습 및 적용** : spring AI framework, LLM 활용 및 프롬프트 엔지니어링, bucket4j(트래픽 제어)

기술

- Spring boot, PostgreSQL + pgvector, bucket4j, Spring AI, JPA, Open AI,Docker
- Next.js, TypeScript, Biome
- Render, Claudeflare, Vercel

프로젝트 회고

정적인 포트폴리오의 한계

문제

기존 포트폴리오는 정보를 한 번에 나열하는 방식이라, 사용자가 원하는 내용을 빠르게 찾기 어렵습니다.
또한, 사용자별로 관심사가 다르기 때문에 정적인 구조만으로는 어필하기 어렵다고 생각하였습니다.

고민

처음에는 단순히 포트폴리오 내용을 더 보기 좋게 배치하는 방향을 생각했지만, 정보량이 많아질수록 사용자가 직접 찾아야 한다는 문제는 그대로 남는다고 판단했습니다.

해결방안

포트폴리오, 이력서, 경력기술서를 데이터화하고, 사용자의 질문에 맞는 내용을 찾아 답변하는 AI기반 포트폴리오 챗봇을 만들고자 하였습니다.

AI가 내 정보를 정확하게 답변하게 만들어야 함

문제

LLM은 제 포트폴리오를 학습한 모델이 아니기 때문에, 별도 처리 없이 질문만 전달하면 **부정확한 답변**을 할 가능성이 있었습니다.

고민

포트폴리오 전체 내용을 프롬프트에 매번 넣는 방식으로 해결하고자 하였으나, 정보가 많아질수록 토큰 비용이 커지고 관련 없는 정보까지 함께 들어가 **답변 품질이 떨어질 수 있다고 생각**했습니다.

해결방안

질문을 임베딩으로 변환한 뒤, **PostgreSQL + pgvector 환경에서 유사도 검색**을 수행하고, 상위 결과만 구성해 LLM에 전달하는 **RAG 패턴이 있다는 것**을 찾아냈습니다.
질문과 관련있는 데이터만 검색해 답변 생성에 활용하도록 설계했습니다.

답변을 자연스럽고 일관성 있게 만들기

문제

초기 응답으로는 3인칭이나, 형식적인 톤으로 출력되는 문제가 있었습니다.

고민

정보만 전달하면 충분할 것이라 생각했지만, 같은 정보라도 어떤 말투와 시점으로 표현하느냐에 따라 인상이 크게 달라졌습니다. 친절하고 일관성 있게 답변을 어떻게 만들어 내야할지 고민되었습니다.

해결방안

시스템 프롬프트에 **역할, 말투, 응답 규칙을 구조화하여 정의**했습니다.
검색 결과, 프로필 정보는 일정한 템플릿으로 넣고, 답변은 1인칭 시점으로 존댓말을 사용하도록 프롬프트를 구성했습니다.

성능과 비용 사이의 균형을 맞추는 문제

문제

LLM 기반 서비스는 사용하는 **모델과 프롬프트 길이에 따라 비용 부담**이 크게 달라졌습니다.
더 높은 성능의 모델을 적용하고 싶었지만, 실제 운영 가능성을 고려하면 비용을 무시할 수 없었습니다.

고민

성능이 좋은 모델을 사용하면 답변 품질은 높아지지만 비용이 급격히 증가하고, 많은 컨텍스트를 포함하면 정확도는 높아질 수 있어도 **토큰 사용량과 응답 시간**이 함께 늘어났습니다.
또한, 개인 프로젝트 특성상 **클라우드 인프라 비용**도 부담이 되었기 때문에, 가능한 한 무료 또는 저비용 서비스를 중심으로 운영 방식을 설계하고자 했습니다.

해결방안

대화 모델로는 **GPT-4o-mini**를 선택하고, 최근 대화 5개만 유지하도록 제한했으며, 검색 결과 역시 **상위 5개만 컨텍스트에 포함**하도록 구성했습니다. 또한 전체 데이터를 매번 프롬프트에 넣지 않고, **RAG 방식으로 필요한 정보만 추출**해 전달함으로써 토큰 사용량을 줄였습니다.
배포는 **프론트엔드에 Vercel**, **백엔드에 Render**를 적용해 비용을 최소화했으며, **Cloudflare 스케줄링**을 통해 저비용 환경에서도 안정적으로 서비스를 유지할 수 있도록 구성했습니다.

https://github.com/Hediar/popo

배운점

- 사용자에게 중요한 것은 많은 정보를 보여주는 것이 아니라, **필요한 정보를 빠르게 전달하는 경험**이 중요합니다.
- AI 서비스의 품질은 모델보다도 **검색 구조와 프롬프트 설계**에 크게 좌우됩니다.
- 개인 프로젝트라도 **비용과 배포 환경**까지 고려해야 실제로 운영 가능한 서비스가 됩니다.

AI 서비스의 핵심은 단순히 LLM을 연결하는 것보다, 데이터를 어떻게 가공하고 검색 가능한 형태로 설계하느냐가 더 중요함을 배웠습니다.
실제로 포트폴리오, 이력서, 경력기술서를 답변에 활용할 수 있도록 정리하고 청크 단위로 구성하는 과정에서 예상보다 많은 시간과 노력이 필요했습니다. 또한, 임베딩 생성과 벡터 유사도 검색은 데이터가 늘어날수록 비용과 성능에 영향을 주기 때문에, 정확도 뿐만 아니라 운영 효율까지 함께 고려해야 했습니다.
데이터 전처리, 검색 구조, 응답 품질, 비용과 성능의 균형을 함께 설계하는 과정을 체감할 수 있었습니다.

로그 모니터링을 위한 경량 로그 시스템 2025. 12 ~ 2026.01

Project

도입 배경

15개 이상의 서버를 운영하다 보니 **로그를 확인**해야 하는 상황이 자주 발생했습니다. 하지만 실제로는 로그가 무엇을 의미하는지 알아야 했고, 매번 서버에 접속해 직접 파일을 열어봐야 했습니다.

이 과정은 로그에 익숙하지 않은 사람들에게는 부담이 컸고, 결국 몇몇 사람만 로그를 확인하는 구조가 되었습니다.
이 문제를 해결하기 위해, **누구나 쉽게 접근할 수 있는 경량 로그 모니터링 도구**를 도입했습니다.

성과

- 운영팀의 **수작업 시간 감소**
- 시간/키워드 기반 검색으로 에러 발생 시점 및 패턴을 즉시 파악 => **대응 속도 향상**

목표

1. 원격 서버의 로그를 새벽 수집한다 => **정보 최신화**, 서버가 활발하지 않는 시간에 수집함으로 **부하 최소화**
2. 검색할 수 있다 => **필요한 내용, 시각**에 발생한 로그 확인 가능
3. 팀원 누구나 웹 UI로 확인할 수 있어야 한다 => **비개발자도 대응 가능하도록** 모니터링 제공

기술

React.js + Express.js + Cron + Opensearch + SFTP

프로젝트 회고

도입 전에는 장애가 나면 대상 **서버 확인 → SSH 접속 → 로그 파일 직접 탐색 흐름**이라 분석 진입장벽이 높았고, 원인 파악까지 시간이 들었습니다.
도입 후에는 웹 UI에서 로그를 검색해 에러 발생 시점/패턴을 빠르게 확인할 수 있게 되었고, 로그에 익숙하지 않은 팀원도 동일한 화면에서 상황 파악에 참여할 수 있게 된 점이 가장 큰 변화였습니다.

구조는 운영 부담을 줄이기 위해 복잡한 파이프라인/메시지 큐 없이 최소 구성으로 설계했습니다.
cron이 ingest job을 실행 → ingest job이 SFTP로 로그 수집 → OpenSearch 색인 → UI는 Express API로 조회하는 형태로, 장애 지점을 최소화했습니다.

운영 로그는 민감할 수 있어 초기에는 테스트 서버 로그만 대상으로 제한하고, 접근 권한/노출 범위를 최소화했습니다.
안정성을 확인한 뒤 점진적으로 대상 서버를 확대할 계획입니다.

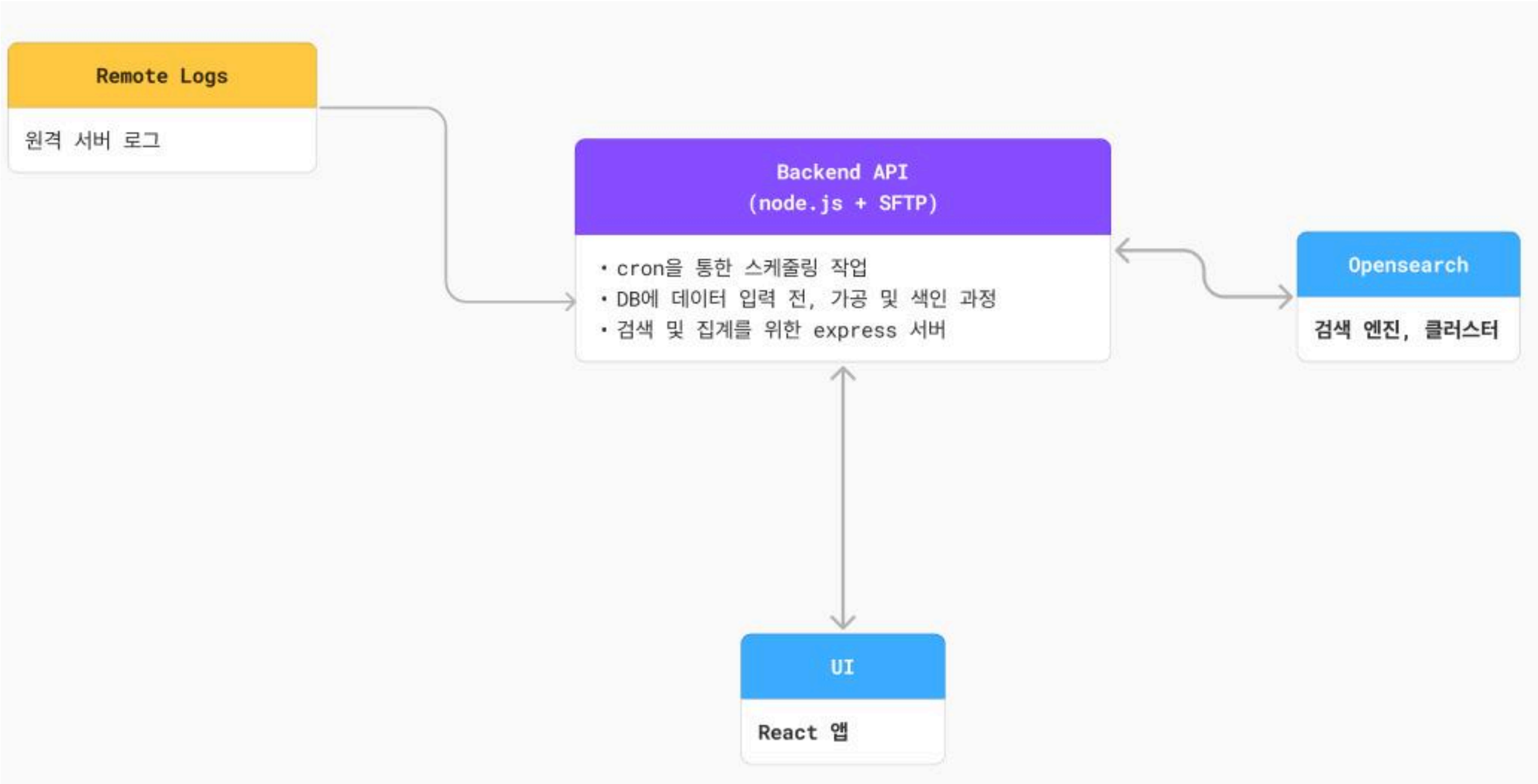
잘한 선택 (Keep)

- 처음부터 운영 환경에 적용하지 않고 **테스트 서버에서 검증**한 점
- rsync/scp 대신 **SFTP 단일 방식**으로 단순화한 점
- ingest job(쓰기)과 API 서버(읽기)를 분리한 구조

아쉬웠던 점(Problem)

- 초기에는 rsync/scp 기반 수집을 시도했지만,
  실행 환경마다 의존성 차이로 문제가 발생했습니다.
  결국 운영 안정성을 위해 SFTP 단일 방식으로 정리하게 되었습니다.
- 또한 로그 포맷이 서버마다 조금씩 달라
  파싱 로직을 유연하게 가져가야 했던 점도 생각보다 까다로웠습니다.

구조



불도저(BDZ) 2025. 12 ~ (진행중)

Project

개요

공영 + 유료 주차장 정보와 불법주정차 단속구역, AI 추천·챗봇, 간편결제를 한 번에 제공하는 서울 시민용 통합 주차 앱

목표

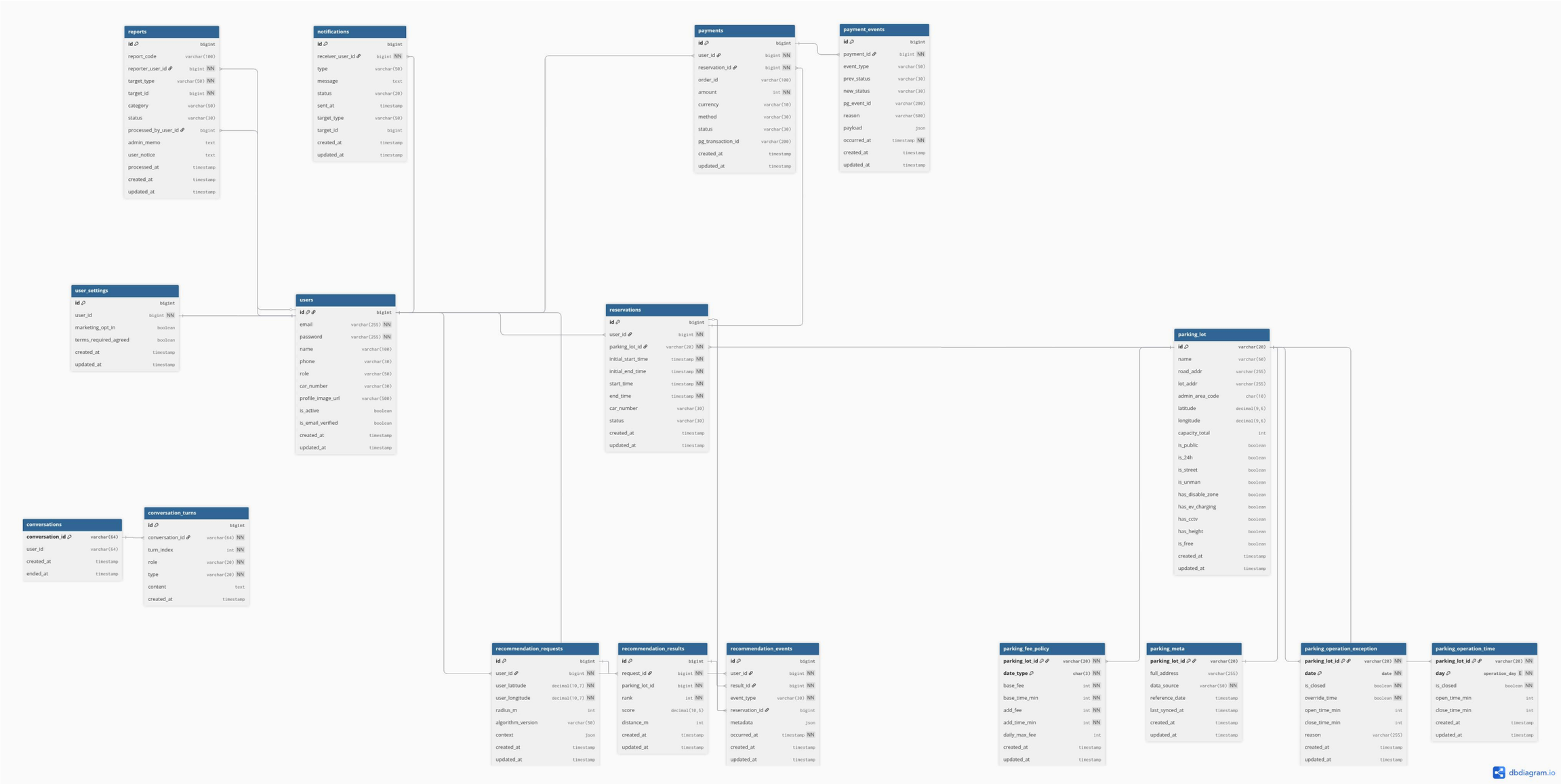
- 시민에게는: 공영 + 유료 주차장 + 단속 정보를 한 번에 보여주고, 검색부터 결제·주차·알림까지 원스톱 제공
- 주차장 관리자에게는: 개인 소유 유료 주차장을 등록·홍보·운영·정산할 수 있는 채널 제공
- 행정에게는: 신고·단속·주차 데이터가 모이는 정책/단속 참고 데이터 인프라 구축

역할

- 백엔드 개발 : Auth, 사용자
- ERD 초안 구성(dbdiagram.io)

기술

Spring boot, Postgresql, Redis, Spring Security, Docker



Auth 기능 배경

모바일 웹/앱 서비스를 함께 제공하는 환경에서 인증의 **신뢰성, 응답속도, 확장성**은 중요합니다.
특히, 모바일 앱은 쿠키 기반 세션 관리가 제한적이고, 네트워크 상태가 불안정한 상황에서도 **일관된 인증 처리가 필요**합니다.

세션 기반 인증은 서버 확장 시 상태 공유 비용이 증가하고, 다중 인스턴스 환경에서 관리 복잡도가 커지는 문제가 있었습니다.
이에 따라 **무상태(Stateless) API**에 적합하고 **모바일 친화적인 JWT 기반 인증 구조**를 선택했으며,
토큰 무효화와 보안 강화를 위해 **Redis를 결합**한 인증 체계를 설계/구현했습니다.

목표

- 보안: BCrypt 해시 기반 비밀번호 저장, 표준 JWT 서명/검증 적용
- 성능/확장성: 액세스 토큰은 무상태 검증, **리프레시 토큰은 Redis로 관리**해 DB 의존 최소화
- 운영 편의: Spring Security 기반 일관된 필터 체인, DB 커넥션/스키마 변경/헬스체크를 표준화
- 유지보수: 설정/환경 분리, 헬스체크/로깅을 통한 관측성 확보

주요 설계 / 구현

- **인증·인가**
  - Spring Security **필터 체인**으로 /api/v1/auth/** 공개, 그 외 엔드포인트 보호
  - BCryptPasswordEncoder, DaoAuthenticationProvider 기반 표준 인증 흐름 구성
  - JwtAuthenticationFilter + JwtTokenProvider로 액세스/리프레시 토큰 발급/검증
- **토큰 저장 전략**
  - **액세스 토큰**: JWT Claims 서명 검증 기반 무상태 처리
  - **리프레시 토큰**: **Redis(refresh:{userId}) + TTL 관리**로 로테이션 및 강제 로그아웃 지원
- **데이터베이스**
  - PostgreSQL + JPA(Hibernate), HikariCP 파라미터 튜닝으로 안정적 커넥션 유지

잘한 점

- **책임 분리**: 토큰 발급/검증, Redis 접근, 도메인 로직 분리로 테스트 용이성 확보
- **보안 기본기**: BCrypt, 토큰 만료/검증, 민감정보 환경변수 분리
- **확장성**: 무상태 액세스 토큰 + 상태적 리프레시 토큰

아쉬운점

- OAuth2 연동까지 확장하지 못해 소셜 로그인 시나리오를 다루지 못했습니다.
- 테스트는 핵심 로직 위주로만 구성했고, 인증 플로우 E2E/보안 시나리오 자동화가 부족했습니다.
- TDD로 설계를 이끌진 못해, 이후 구현과정에서 시나리오 기반 테스트를 구현하는 방식으로 개선할 계획입니다.

회고

인증 기능을 구현하며 보안/성능/운영 안정성의 균형이 중요하다는 것을 체감했습니다.
모바일 환경에서는 JWT 기반 무상태 인증이 확장성과 응답 속도 면에서 적합했고, Redis를 활용한 리프레시 토큰 관리가 운영 측면에서 효과적이었습니다.
다음에는 테스트 기반 설계와 OAuth2 확장을 고려한 구조로 인증 시스템을 발전시키고자 합니다.